assignment 3 - NLPROC-ALV-B

Anja Kieburg, Friedemann Decker

Task 2 Programming

Task 2 TIGER Corpus

Korpus TIGER 2.2.doc (April 2018) from: https://www.ims.uni-stuttgart.de/en/research/resources/corpora/tiger/

The TIGER Corpus consists of approximate 900,000 tokens (50,000 sentences) of German newspaper text, taken from the Frankfurter Rundschau. The corpus was semi-automatically POS-tagged and annotated with syntactic structure. Moreover, it contains morphological and lemma information for terminal nodes

In [3]:
```
import pandas as pd
import numpy as np
```

Step 1: Import TIGER XML into Pandas, using only the entries under `s=Sätze`

In [4]:
```
tiger_corpus = pd.read_xml("tiger_release_aug07.corrected.16012013.xml",
                    xpath="//s/graph/terminals/t",
                    elems_only=False,
                    encoding="ISO-8859-1")
```

In [5]:
```
len_tiger_corpus = len(tiger_corpus)
len(tiger_corpus)
```

Out[5]: 888238

In [6]:
```
tiger_corpus.head()
```

Out[6]:

	id	word	lemma	pos	morph	case	number	gender	person	degree	tense	mood	secedge
0	s1_1	``	--	$(	--	--	--	--	--	--	--	--	NaN
1	s1_2	Ross	Ross	NE	Nom.Sg.Masc	Nom	Sg	Masc	--	--	--	--	NaN
2	s1_3	Perot	Perot	NE	Nom.Sg.Masc	Nom	Sg	Masc	--	--	--	--	NaN
3	s1_4	wäre	sein	VAFIN	3.Sg.Past.Subj	--	Sg	--	3	--	Past	Subj	NaN
4	s1_5	vielleicht	vielleicht	ADV	--	--	--	--	--	--	--	--	NaN

Task 2 A) tag t that maximizes p(t|w)

Step 2: Required fields: Import "word" and "pos" into the DataFrame

In [7]:
```
words = tiger_corpus[["word", "pos"]]
words.head()
```

Out[7]:

	word	pos
0	``	$(
1	Ross	NE
2	Perot	NE
3	wäre	VAFIN
4	vielleicht	ADV

Step 3: Count the number of unique combinations of "word" AND "pos" = frequency of occurrence in the corpus

In [8]:
```
word_pos_freq = words[["word", "pos"]].value_counts()
# Set -> df
words = word_pos_freq.reset_index(name="pos_freq")
words.head()
```

Out[8]:

	word	pos	pos_freq
0	,	$,	43656
1	.	$.	41057
2	der	ART	25286
3	die	ART	21209
4	und	KON	16281

Step 4: Convert "word" to lowercase so that different parts of speech (POS) can be grouped into a "word_lower" category

In [9]:
```
word_lower = words["word"].str.lower()
words.insert(1, "word_lower", word_lower)
words.head()
```

Out[9]:

	word	word_lower	pos	pos_freq
0	,	,	$,	43656
1	.	.	$.	41057
2	der	der	ART	25286
3	die	die	ART	21209
4	und	und	KON	16281

Step 5: Determine the number of words per "word_lower" category and enter it as "word_freq" in each row to calculate p(t|w).

In [10]:
```
word_freq = words.groupby("word_lower")["pos_freq"].transform("sum")
words.insert(4, "word_freq", word_freq)
words.head()
```

Out[10]:

	word	word_lower	pos	pos_freq	word_freq
0	,	,	$,	43656	43659
1	.	.	$.	41057	41057
2	der	der	ART	25286	29662
3	die	die	ART	21209	30024
4	und	und	KON	16281	16786

Step 6: Calculate p_pos = p(t|w)

In [11]:
```
p_pos = words["pos_freq"] / words["word_freq"]
words.insert(5, "p_pos", p_pos)
words.head()
```

Out[11]:

	word	word_lower	pos	pos_freq	word_freq	p_pos
0	,	,	$,	43656	43659	0.999931
1	.	.	$.	41057	41057	1.000000
2	der	der	ART	25286	29662	0.852471
3	die	die	ART	21209	30024	0.706402
4	und	und	KON	16281	16786	0.969915

Step 7: Keep only p_pos_max values in the DataFrame

In [12]:
```
words["max_p_pos"] = words.groupby("word_lower")["p_pos"].transform("max")
words = words[words["p_pos"] == words["max_p_pos"]]

words.head()
```

Out[12]:

	word	word_lower	pos	pos_freq	word_freq	p_pos	max_p_pos
0	,	,	$,	43656	43659	0.999931	0.999931
1	.	.	$.	41057	41057	1.000000	1.000000
2	der	der	ART	25286	29662	0.852471	0.852471
3	die	die	ART	21209	30024	0.706402	0.706402
4	und	und	KON	16281	16786	0.969915	0.969915

Step 8: Delete columns that aren't needed (clean up). Rename the columns according to the assignment.

In [13]:
```
words = words.drop(columns = ["word_lower", "p_pos"])
```

In [14]:
```
words = words.rename(columns={'word': 'w', 'pos': 't', 'max_p_pos': 'p'})
```

Result: Computation of the tag **t** for each word **w** that maximizes **p(t|w)**. List of the first 30 values:

In [15]:
```
words.head(30)
```

Out[15]:

	w	t	pos_freq	word_freq	p
0	,	$,	43656	43659	0.999931
1	.	$.	41057	41057	1.000000
2	der	ART	25286	29662	0.852471
3	die	ART	21209	30024	0.706402
4	und	KON	16281	16786	0.969915
5	in	APPR	13711	15102	0.907893
6	den	ART	9486	9955	0.952888
7	''	$(	8811	8811	1.000000
8	``	$(	8809	8809	1.000000
9	von	APPR	8182	9033	0.905790
10	mit	APPR	6035	6872	0.878201
11	des	ART	5903	5920	0.997128
12	für	APPR	5859	6328	0.925885
13	sich	PRF	5585	5595	0.998213
14	auf	APPR	5577	6301	0.885098
16	nicht	PTKNEG	5143	5294	0.971477
17	im	APPRART	5137	5899	0.870826
18	das	ART	4742	7977	0.594459
19	dem	ART	4492	5070	0.885996
20	ist	VAFIN	4420	4464	0.990143
21	zu	PTKZU	4402	7333	0.600300
22	eine	ART	4179	4763	0.877388
23	)	$(	4177	4177	1.000000
24	(	$(	4159	4159	1.000000
25	ein	ART	3904	4995	0.781582
26	auch	ADV	3444	4068	0.846608
27	es	PPER	3105	3902	0.795746
29	daß	KOUS	3054	3257	0.937673
30	:	$.	2962	3051	0.970829
31	an	APPR	2900	3852	0.752856

Appendix: Meaning of the Abbreviations Used for Parts of Speech (POS)

In [294…

```
tiger_corpus_pos_abr = pd.read_xml("tiger_release_aug07.corrected.16012013.xml",
                  xpath="//head/annotation/feature/value",
                  elems_only=False,
                  encoding="ISO-8859-1")
```

In [295…

```
tiger_corpus_pos_abr = tiger_corpus_pos_abr.iloc[:-644]
tiger_corpus_pos_abr
```

Out[295...

	name	value
0	$(	Sonstige Satzzeichen; satzintern
1	$,	Komma
2	$.	Satzbeendende Interpunktion
3	--	<Nicht zugeordnet>
4	ADJA	Attributives Adjektiv
5	ADJD	Adverbiales oder prädikatives Adjektiv
6	ADV	Adverb
7	APPO	Postposition
8	APPR	Präposition; Zirkumposition links
9	APPRART	Präposition mit Artikel
10	APZR	Zirkumposition rechts
11	ART	Bestimmter oder unbestimmter Artikel
12	CARD	Kardinalzahl
13	FM	Fremdsprachliches Material
14	ITJ	Interjektion
15	KOKOM	Vergleichspartikel, ohne Satz
16	KON	Nebenordnende Konjunktion
17	KOUI	Unterordnende Konjunktion mit zu und Infinitiv
18	KOUS	Unterordnende Konjunktion mit Satz
19	NE	Eigennamen
20	NN	Normales Nomen
21	NNE	Kombination aus Nomen und Eigenname
22	PDAT	Attribuierendes Demonstrativpronomen
23	PDS	Substituierendes Demonstrativpronomen
24	PIAT	Attribuierendes Indefinitpronomen
25	PIS	Substituierendes Indefinitpronomen
26	PPER	Ersetzbares Personalpronomen
27	PPOSAT	Attribuierendes Possessivpronomen
28	PPOSS	Substituierendes Possessivpronomen
29	PRELAT	Attribuierendes Relativpronomen
30	PRELS	Substituierendes Relativpronomen
31	PRF	Reflexives Personalpronomen
32	PROAV	Pronominaladverb
33	PTKA	Partikel bei Adjektiv oder Adverb
34	PTKANT	Antwortpartikel
35	PTKNEG	Negationspartikel
36	PTKVZ	Abgetrennter Verbzusatz
37	PTKZU	zu vor Infinitiv
38	PWAT	Attribuierendes Interrogativpronomen
39	PWAV	Adverbiales Interrogativ- oder Relativpronomen
40	PWS	Substituierendes Interrogativpronomen

	name	value
41	TRUNC	Kompositions-Erstglied
42	UNKNOWN	Unbekanntes Tag aus Einlesen aus Korpusdatei
43	VAFIN	Finites Verb, aux
44	VAIMP	Imperativ, aux
45	VAINF	Infinitiv, aux
46	VAPP	Partizip Perfekt, aux
47	VMFIN	Finites Verb, modal
48	VMINF	Infinitiv, modal
49	VMPP	Partizip Perfekt, modal
50	VVFIN	Finites Verb, voll
51	VVIMP	Imperativ, voll
52	VVINF	Infinitiv, voll
53	VVIZU	Infinitiv mit zu, voll
54	VVPP	Partizip Perfekt, voll
55	XY	Nichtwort, Sonderzeichen

Task 2 B) - HMM tagger

Step 1) Split the corpus into a training set and test set.

In [53]:
```
# SOLL Anzahl Worte
len_train = int(len_tiger_corpus * 0.95) + 1 # Manual len adjustment due to full sentence split
len_test = len_tiger_corpus - len_train
print(f"Words in train set calc: {len_train}")
print(f"Words in test set calc: {len_test}")
```

```
Words in train set calc: 843827
Words in test set calc: 44411
```

In [205…
```
#IST Anzahl Worte
train = tiger_corpus.iloc[:len_train]
train = train[["word", "pos"]]
test = tiger_corpus.iloc[len_train:].set_index("id")
print(f"Words in train set act: {len(train)}")
print(f"Words in test set act: {len(test)}")
test.head(15)
```

```
Words in train set act: 843827
Words in test set act: 44411
```

Out[205…

id	word	lemma	pos	morph	case	number	gender	person	degree	ter
s48037_1	Weithin	weithin	ADV	--	--	--	--	--	--	
s48037_2	reicht	reichen	VVFIN	3.Sg.Pres.Ind	--	Sg	--	3	--	P
s48037_3	die	der	ART	Nom.Sg.Fem	Nom	Sg	Fem	--	--	
s48037_4	Sicht	Sicht	NN	Nom.Sg.Fem	Nom	Sg	Fem	--	--	
s48037_5	in	in	APPR	--	--	--	--	--	--	
s48037_6	die	der	ART	Acc.Sg.Fem	Acc	Sg	Fem	--	--	
s48037_7	Manila-Bucht	Manila-Bucht	NE	Acc.Sg.Fem	Acc	Sg	Fem	--	--	
s48037_8	,	--	$,	--	--	--	--	--	--	
s48037_9	die	der	PRELS	Nom.Sg.Fem	Nom	Sg	Fem	--	--	
s48037_10	wegen	wegen	APPR	--	--	--	--	--	--	
s48037_11	ihres	ihr	PPOSAT	Gen.Sg.Masc	Gen	Sg	Masc	--	--	
s48037_12	Sonnenuntergangs	Sonnenuntergang	NN	Gen.Sg.Masc	Gen	Sg	Masc	--	--	
s48037_13	weltberühmt	weltberühmt	ADJD	Pos	--	--	--	--	Pos	
s48037_14	ist	sein	VAFIN	3.Sg.Pres.Ind	--	Sg	--	3	--	P
s48037_15	.	--	$.	--	--	--	--	--	--	

Step 2) Counting of POS tags, POS tag - POS tag follower, word - POS tag combination

From the training set, count how often each (i) POS tag occurs, how often one (ii) POS tag follows another and how often each (iii) word occurs with each POS tag.

In [55]:
```
# Step 2 (i) Occurence of POS tags
train_pos_count = train[["pos"]].value_counts()
# Set -> df
train_pos_count = train_pos_count.reset_index(name="pos_count")
train_pos_count.head()
```

Out[55]:

	pos	pos_count
0	NN	175113
1	ART	92486
2	APPR	71802
3	ADJA	51747
4	NE	49643

In [325…
```
# Step 2 (ii) POS tag follows another
# Testset mit Follower bauen
# Follower Spalte bauen, alle Zeilen um 1 Zeile nach oben, letzte Zeile = "NaN"
train_foll = pd.DataFrame()
train_foll[["word_foll", "pos_foll"]] = train[["word", "pos"]]
train_foll = train_foll.iloc[1:].reset_index(drop=True)
train_foll.loc[len(train_foll)] = np.nan

# Follower Spalte zu Testset hinzufügen
train = pd.concat([train, train_foll], axis=1)
```

In [327…
```
train
```

Out[327…

	word	pos	word_foll	pos_foll
0	``	$(	Ross	NE
1	Ross	NE	Perot	NE
2	Perot	NE	wäre	VAFIN
3	wäre	VAFIN	vielleicht	ADV
4	vielleicht	ADV	ein	ART
...	...	...	...	...
843822	grüne	ADJA	Lunge	NN
843823	Lunge	NN	der	ART
843824	der	ART	Zehn-Millionen-City	NN
843825	Zehn-Millionen-City	NN	.	$.
843826	.	$.	NaN	NaN

843827 rows × 4 columns

In [326…

```
train_pos_pos_foll_count = pd.crosstab(train["pos"], train["pos_foll"])
train_pos_pos_foll_count #debug print
```

Out[326...

pos_foll pos	$(	$,	$.	ADJA	ADJD	ADV	APPO	APPR	APPRART	APZR	...	VAPP	VMFIN	VMINF	V
$(	668	2225	2646	1792	786	1301	1	2155	406	1	...	44	214	2	
$,	558	3	0	1737	594	2053	0	3116	431	0	...	1	817	0	
$.	3273	13	5	838	1045	4183	0	5654	1212	0	...	0	75	0	
ADJA	846	720	66	2239	23	54	0	127	20	2	...	2	10	0	
ADJD	663	1460	1651	1463	770	640	0	2119	306	1	...	114	92	0	
ADV	534	910	689	1805	3009	4468	0	5610	1047	16	...	15	501	0	
APPO	11	6	9	3	12	27	0	32	6	0	...	0	12	0	
APPR	793	8	13	5245	685	1626	0	250	282	0	...	0	0	0	
APPRART	139	1	1	2992	26	43	0	6	1	0	...	0	0	0	
APZR	16	16	10	11	14	21	0	30	8	0	...	0	17	0	
ART	1843	20	0	22603	722	1008	0	1299	351	0	...	0	0	0	
CARD	662	329	251	698	103	229	0	907	101	38	...	3	63	0	
FM	244	58	45	1	10	11	0	29	11	0	...	0	5	0	
ITJ	2	6	4	0	0	1	0	0	0	0	...	0	0	0	
KOKOM	28	0	0	80	82	229	0	284	107	0	...	1	3	0	
KON	193	34	32	1505	680	1744	0	1437	288	0	...	0	144	0	
KOUI	8	1	0	41	25	72	0	96	15	0	...	0	0	0	
KOUS	59	15	3	135	59	376	0	389	89	0	...	0	4	0	
NE	5497	5438	2694	927	663	1378	40	3634	1317	35	...	20	648	0	
NN	8788	11965	10712	3583	3910	5959	190	31421	5285	171	...	143	2518	0	
NNE	0	0	0	0	0	0	0	0	0	0	...	0	0	0	
PDAT	53	9	0	293	16	22	0	20	5	0	...	0	0	0	
PDS	25	172	41	74	60	274	2	235	38	0	...	0	174	0	
PIAT	52	20	8	903	30	28	0	21	5	0	...	0	0	0	
PIS	77	223	125	53	151	344	2	712	93	0	...	2	167	0	
PPER	129	358	215	202	453	1636	5	1692	312	3	...	4	506	0	
PPOSAT	99	2	0	1066	25	67	0	32	13	0	...	0	0	0	
PPOSS	0	2	1	0	0	0	0	3	0	0	...	0	0	0	
PRELAT	4	0	0	40	3	1	0	3	0	0	...	0	0	0	
PRELS	49	26	0	100	224	744	6	1269	247	3	...	0	1	0	
PRF	48	82	45	108	322	821	0	1420	221	10	...	0	0	0	
PROAV	45	499	76	111	151	367	0	318	54	54	...	0	273	0	
PTKA	3	0	0	58	280	27	0	0	1	0	...	0	0	0	
PTKANT	13	43	10	0	1	0	0	1	0	0	...	0	0	0	
PTKNEG	49	160	196	103	630	1225	0	621	119	0	...	3	9	0	
PTKVZ	190	1570	2469	7	1	3	0	100	11	0	...	1	0	0	
PTKZU	13	0	0	51	0	0	0	0	0	0	...	0	0	171	
PWAT	4	0	0	21	0	0	0	1	0	0	...	0	0	0	
PWAV	8	7	13	21	103	132	0	63	27	4	...	0	51	0	
PWS	6	6	3	10	20	102	0	93	20	0	...	0	22	0	

pos_foll / pos	$(	$,	$.	ADJA	ADJD	ADV	APPO	APPR	APPRART	APZR	...	VAPP	VMFIN	VMINF	\
TRUNC	42	92	0	3	0	1	0	7	2	0	...	0	0	0	
VAFIN	578	2286	2840	270	984	2666	0	2409	751	0	...	1	1	0	
VAIMP	0	0	0	0	2	0	0	0	0	0	...	0	0	0	
VAINF	92	689	1399	0	0	0	0	29	3	0	...	0	609	34	
VAPP	19	280	611	0	0	0	0	3	2	0	...	0	0	0	
VMFIN	165	766	1159	95	215	819	0	771	144	0	...	0	0	0	
VMINF	23	147	294	0	0	0	0	3	0	0	...	0	0	0	
VMPP	1	0	2	0	0	0	0	0	0	0	...	0	0	0	
VVFIN	642	4449	3719	419	569	2599	0	2950	689	0	...	0	0	0	
VVIMP	4	0	5	6	1	30	0	2	1	0	...	0	0	0	
VVINF	673	2785	5272	11	5	12	0	103	5	0	...	8	1466	270	
VVIZU	81	389	793	0	2	0	0	5	1	0	...	2	0	0	
VVPP	559	3016	5100	9	3	19	0	305	38	0	...	910	5	1	
XY	308	14	7	15	3	4	0	16	6	0	...	0	0	0	

54 rows × 54 columns

In [193...

```
# Step 2 (iii) Occurence of word with each POS tag
train_word_pos_count = train[["word", "pos"]].value_counts()
# Set -> df
train_word_pos_count = train_word_pos_count.reset_index(name="word_pos_count")
# Gesamtanzahl pro POS tag zu jeder Kombination word_pos in neuer Spalte ergänzen
train_word_pos_count = train_word_pos_count.merge(train_pos_count[["pos", "pos_count"]], on="pos", how="
# Gesamtzahl pro word zu jeder Kombination word_pos in neuer Spalte ergänzen
train_word_count = train[["word"]].value_counts()
# Set -> df
train_word_count = train_word_count.reset_index(name="word_count")
train_word_pos_count = train_word_pos_count.merge(train_word_count[["word", "word_count"]], on="word", h

train_word_pos_count
```

Out[193...

	word	pos	word_pos_count	pos_count	word_count
0	,	$,	41320	41320	41321
1	.	$.	39017	43234	39017
2	der	ART	24084	92486	25482
3	die	ART	20127	92486	23213
4	und	KON	15366	20832	15367
...	...	...	...	...	...
90501	erfrischt	VVPP	1	16790	1
90502	durchatmen	VVINF	1	12757	1
90503	Großstadtdschungel	NN	1	175113	1
90504	Metro-Manila	NE	1	49643	1
90505	Zehn-Millionen-City	NN	1	175113	1

90506 rows × 5 columns

Step 3 Compute the (i) transition and (ii) observation probabilities of the HMM tagger from this.

In [184…
```
# Step 3(i) transition probabilities
# Zeilensummen von trai_pos_pos_foll_count ermitteln, train_pos_foll_sum ist ein Set
train_pos_foll_sum = train_pos_pos_foll_count.sum(axis=1)
# Jeden Spaltenwert durch Zeilensumme teilen
train_pos_trans_prob = train_pos_pos_foll_count.div(train_pos_foll_sum, axis=0)
#train_pos_trans_prob    #debug print
```

In [198…
```
# Step 3(ii) observation probabilities (Gibt an, mit welcher Wahrscheinlichkeit ein Wort als bestimmte W
train_word_pos_count["tag_prob"] = train_word_pos_count["word_pos_count"] / train_word_pos_count["word_c
train_word_pos_count["word_obs_prob"] = train_word_pos_count["word_pos_count"] / train_word_pos_count["p
train_word_pos_count["init_viterbi"] = train_word_pos_count["tag_prob"] * train_word_pos_count["word_obs
train_word_pos_count
```

Out[198…

	word	pos	word_pos_count	pos_count	word_count	tag_prob	word_obs_prob	init_viterbi
0	,	$,	41320	41320	41321	0.999976	1.000000	0.999976
1	.	$.	39017	43234	39017	1.000000	0.902461	0.902461
2	der	ART	24084	92486	25482	0.945138	0.260407	0.246120
3	die	ART	20127	92486	23213	0.867057	0.217622	0.188691
4	und	KON	15366	20832	15367	0.999935	0.737615	0.737567
...	...	...	...	...	...	...	...	...
90501	erfrischt	VVPP	1	16790	1	1.000000	0.000060	0.000060
90502	durchatmen	VVINF	1	12757	1	1.000000	0.000078	0.000078
90503	Großstadtdschungel	NN	1	175113	1	1.000000	0.000006	0.000006
90504	Metro-Manila	NE	1	49643	1	1.000000	0.000020	0.000020
90505	Zehn-Millionen-City	NN	1	175113	1	1.000000	0.000006	0.000006

90506 rows × 8 columns

Then implement the Viterbi algorithm so you can decode a test sentence.

Step 4: Create a Viterbi table that stores, for each position in the sentence and each possible tag, the probability of the most probable path ending in that tag.

In [218…
```
# Get test sentence out of test
s_no = 48037
s_id = "s" + str(s_no) + "_"
sentc = test[test.index.str.startswith(s_id)]
sentc = sentc["word"]
sentc
```

Out[218…
```
id
s48037_1              Weithin
s48037_2               reicht
s48037_3                  die
s48037_4               Sicht
s48037_5                  in
s48037_6                  die
s48037_7         Manila-Bucht
s48037_8                    ,
s48037_9                  die
s48037_10               wegen
s48037_11               ihres
s48037_12    Sonnenuntergangs
s48037_13         weltberühmt
s48037_14                 ist
s48037_15                   .
Name: word, dtype: str
```

In [323…
```
# Spaltennahme = Satz aus Testset, Zeilenindex = POS-Einträge aus TIGER-Korpus
viterbi = pd.DataFrame(columns=sentc, index=tiger_corpus_pos_abr["name"])
```

Step 5: Initialize the Viterbi table for the first word using the initial tag probabilities and the observation probability of that word.

Anmerkung: Für diese Übung wird nicht berücksichtigt, dass für das erste Wort eines Satzes eigentlich ein Satz-Start-Token <s> benötigt werden würde.

In [272…
```
# Handeingabe zu Testzwecken (Trainingsset)
# Hole initiale tag Wahrscheinlichkeiten für aktuelles 'word' vom Training Korpus
init_tag_prob = train_word_pos_count[train_word_pos_count["word"] == "Weiterhin"]
init_tag_prob
```

Out[272…

	word	pos	word_pos_count	pos_count	word_count	tag_prob	word_obs_prob	init_viterbi
28530	Weiterhin	ADV	2	37366	2	1.0	0.000054	0.000054

In [318…
```
# Handeingabe zu Testzwecken (Testset)
test[test["word"] == "Weiterhin"]
```

Out[318…

id	word	lemma	pos	morph	case	number	gender	person	degree	tense	mood	secedge

In [322…
```
# Mit KI-Untertützung ertellt, siehe Dokument KI01-Pandas-Zuweisung für Viterbi-Matrizen.pdf

i = 0
word = viterbi.columns[i]
# Hole initiale tag Wahrscheinlichkeiten für erstes 'word' vom Training Korpus
init_tag_prob = train_word_pos_count[train_word_pos_count["word"] == word]
neue_werte = init_tag_prob.set_index('pos')['init_viterbi']
# Direktes Mapping: reindex passt die Werte an den festen viterbi-Index an
viterbi.iloc[:, i] = neue_werte.reindex(viterbi.index, fill_value=0.0)
viterbi_rec = viterbi

# KI-Unterstützung ENDE

viterbi
```

Out[322...

word	Weithin	reicht	die	Sicht	in	die	Manila-Bucht	,	die	wegen	ihres	Sonnenuntergangs	welt
name													
$(	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
$,	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
$.	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
--	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
ADJA	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
ADJD	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
ADV	0.000054	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
APPO	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
APPR	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
APPRART	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
APZR	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
ART	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
CARD	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
FM	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
ITJ	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
KOKOM	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
KON	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
KOUI	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
KOUS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NE	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NN	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NNE	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PDAT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PDS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PIAT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PIS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PPER	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PPOSAT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PPOSS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PRELAT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PRELS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PRF	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PROAV	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PTKA	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PTKANT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PTKNEG	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PTKVZ	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PTKZU	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PWAT	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
PWAV	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

word	Weithin	reicht	die	Sicht	in	die	Manila-Bucht	,	die	wegen	ihres	Sonnenuntergangs	welt
name													
PWS	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
TRUNC	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
UNKNOWN	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VAFIN	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VAIMP	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VAINF	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VAPP	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VMFIN	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VMINF	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VMPP	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VVFIN	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VVIMP	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VVINF	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VVIZU	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
VVPP	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
XY	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

Step 6: Process the remaining words from left to right.

Step 7: For each word, update the Viterbi table by considering all possible tags and the probabilities of the most probable paths leading to them.

Step 8: Store both the best score and the preceding tag that achieved it (the backpointer).

Step 9: After filling the table, backtrack from the last word to find the most probable sequence of tags.

Step 10: Run your algorithm on the test set. Report its error rate and compare its performance to the most frequent tag baseline.

In [ ]: